

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Para el desarrollo del ciclo 2 definimos seis mini-ciclos, cada uno enfocado en una parte puntual del problema para no mezclar cambios grandes al mismo tiempo y reducir el riesgo de dañar la lógica que ya funcionaba del ciclo 1.

Mini-ciclo 1. Preparación del modelo para ciclo 2.

En este mini-ciclo se separó el número lógico del objeto de su tamaño físico, ya que en el ciclo 1 ambos estaban representados por size, pero para el ciclo 2 ya no era suficiente. Esta separación fue necesaria porque ahora el sistema debía soportar objetos identificados por tipo y número, además de un nuevo constructor que crea tazas automáticamente con numeración lógica consecutiva pero tamaños impares. También se agregaron nuevas formas de búsqueda en Tower para encontrar objetos por tipo y número, lo que dejó lista la base para implementar swap, cover y swapToReduce.

Mini-ciclo 2. Implementación del constructor Tower(int cups).

Aquí se implementó el nuevo constructor solicitado por el enunciado. La torre se crea automáticamente con solo tazas, numeradas desde 1 hasta cups, pero con tamaños físicos impares 1, 3, 5, 7.... Este mini-ciclo se justificó porque era una funcionalidad nueva del ciclo 2 y además requería validar que la torre inicial quedara correctamente ordenada desde la taza más grande hasta la más pequeña. En este punto también se creó TowerC2Test para empezar a validar este comportamiento con pruebas unitarias independientes del ciclo 1.

Mini-ciclo 3. Implementación de swap(String[] o1, String[] o2).

En este mini-ciclo se añadió la funcionalidad para intercambiar dos objetos de la torre a partir de su tipo y número. Se justificó como un mini-ciclo independiente porque ya implicaba modificar la estructura interna de la torre y recalculas su estado visual y lógico. También fue necesario reforzar pruebas para casos válidos e inválidos, por ejemplo cuando alguno de los objetos no existía o cuando se intentaba intercambiar un objeto consigo mismo.

Mini-ciclo 4. Implementación de cover().

Se desarrolló el método que reorganiza la torre para dejar cada taza seguida por su tapa cuando ambas existan. Este mini-ciclo se trató por separado porque no solo implicaba reorganizar la lista de objetos, sino también recalculas posiciones, relaciones de pareja y conservar los objetos que no encontraban tapa o taza correspondiente. Además, cover() afectó directamente la visualización, por lo que también requirió validación lógica y visual.

Mini-ciclo 5. Implementación de swapToReduce().

Aquí se implementó la consulta de un intercambio que reduzca la altura actual de la torre sin modificar su estado final. Se justificó como un mini-ciclo aparte porque ya no era solo una operación sobre la torre, sino una simulación temporal de posibles intercambios para comparar alturas. Durante este proceso también se corrigió el método swap, ya que aparecieron errores por intentos de intercambio con objetos inexistentes, lo cual llevó a reforzar validaciones internas.

Mini-ciclo 6. Cierre visual y validación del ciclo 2.

En el último mini-ciclo se reforzó la parte visual del simulador, en especial la representación

de las tazas tapadas y el comportamiento de `cover()` y `swapToReduce()` en pantalla. Además, se realizaron pruebas de aceptación separadas para presentar mejor las funcionalidades del ciclo 2. Este mini-ciclo fue importante porque permitió validar no solo la lógica, sino también que el comportamiento observado en el canvas correspondiera con lo solicitado en el enunciado.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

Actualmente el proyecto se encuentra con el ciclo 2 terminado a nivel funcional, ya que los seis mini-ciclos definidos fueron desarrollados y probados. En este momento el sistema ya permite:

- crear una torre usando `Tower(int cups)`,
- intercambiar objetos con `swap(...)`,
- reorganizar la torre con `cover()`,
- consultar un intercambio que reduzca la altura con `swapToReduce()`,
- validar estos comportamientos mediante pruebas unitarias en `TowerC2Test`,
- y observar el comportamiento visual mediante pruebas de aceptación.

Consideramos que el estado actual del proyecto es estable para pasar al ciclo 3 porque ya no estamos en etapa de preparación de base, sino en una etapa en la que las funcionalidades del ciclo 2 quedaron integradas entre sí y funcionando tanto en pruebas lógicas como visuales. Aunque durante el desarrollo se hicieron varias correcciones, especialmente en validaciones y en la parte de visualización, esas dificultades fueron resueltas dentro del mismo ciclo y no quedaron como bloqueos abiertos.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

- Yeray: 11 horas
- Andrés: 11 horas

4. ¿Cuál consideran que fue el mayor logro? ¿Por qué?

Consideramos que el mayor logro del ciclo 2 fue haber logrado integrar correctamente la parte lógica con la parte visual del simulador sin romper el comportamiento base del ciclo 1. Esto fue importante porque no solo se añadieron nuevas operaciones como `swap`, `cover` y `swapToReduce`, sino que también se tuvo que reorganizar parte del diseño interno para soportarlas.

Otro aspecto valioso fue haber separado correctamente el número lógico del tamaño físico, ya que este cambio permitió que el proyecto creciera de forma más coherente hacia lo que pide el ciclo 2. Aunque ese ajuste parece pequeño, realmente fue una decisión clave de diseño, porque facilitó la implementación de las nuevas funcionalidades y dejó el proyecto mejor preparado para seguir avanzando.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue manejar correctamente la coherencia entre la estructura interna de la torre y su representación visual. En varias partes del ciclo 2 el sistema parecía funcionar desde el punto de vista lógico, pero al verlo en el canvas se detectaban comportamientos que no eran correctos, por ejemplo reordenamientos inesperados, tazas que seguían actuando como contenedores cuando ya estaban cerradas o casos en los que la validación de intercambio generaba errores por índices inválidos.

Para resolver esto, primero se decidió trabajar por mini-ciclos, de manera que cada cambio tuviera un alcance controlado. Además, se reforzaron las pruebas unitarias en TowerC2Test, se añadieron pruebas de aceptación separadas para observar el comportamiento visual y se corrigieron métodos específicos como swap, cover y la lógica de posicionamiento. En general, la solución no fue agregar muchas funcionalidades al tiempo, sino ir corrigiendo el modelo paso a paso hasta que la lógica y la visualización coincidieran.

6. ¿Qué hicieron bien como equipo? ¿A qué se comprometen a hacer para mejorar los resultados?

Como equipo consideramos que hicimos bien la división del problema en mini-ciclos, porque eso nos ayudó a no desordenar el proyecto y a mantener un control más claro sobre qué estábamos cambiando en cada momento. También fue positivo que fuimos corrigiendo errores apenas aparecían, especialmente cuando las pruebas mostraban que un cambio afectaba algo que antes funcionaba.

Otro punto fuerte fue la combinación entre pruebas lógicas y validación visual. No nos quedamos solo con “los tests pasan”, sino que también verificamos cómo se comportaba la torre en pantalla, lo cual fue importante en un proyecto con componente gráfica.

Como compromiso de mejora, consideramos que para el siguiente ciclo debemos:

- dejar más claras las reglas del modelo antes de programar,
- documentar mejor las decisiones de diseño desde el inicio,
- ejecutar pruebas de regresión con más frecuencia,
- y registrar mejor el tiempo invertido por cada integrante para que la retrospectiva quede más precisa.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿por qué?

La práctica XP que consideramos más útil fue la de trabajar en iteraciones pequeñas apoyadas por pruebas frecuentes. En este ciclo fue especialmente importante porque muchas funcionalidades dependían unas de otras, y hacer cambios grandes de una sola vez habría hecho mucho más difícil identificar dónde estaba el error cuando algo fallaba.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Para este ciclo usamos principalmente los documentos del proyecto y algunas fuentes externas para reforzar la parte técnica. Las más útiles fueron las páginas oficiales de Oracle Java, porque nos ayudaron a entender mejor cómo aplicar polimorfismo, sobrescritura de métodos y documentación con Javadoc dentro del código. También usamos un ejemplo de Stack Overflow para contrastar dudas puntuales de implementación y aterrizar mejor algunos casos prácticos.

La referencia que consideramos más útil fue la de Oracle sobre polymorphism, porque fue la que más nos ayudó a justificar por qué parte de la lógica debía ir en la jerarquía StackItem y no quedarse toda concentrada en Tower. La de Javadoc también fue importante porque nos sirvió para documentar los métodos de forma más ordenada y consistente con Java.

Referencias:

- Oracle. (s. f.). *Overriding and hiding methods*. The Java Tutorials. Recuperado el 14 de marzo de 2026, de <https://docs.oracle.com/javase/tutorial/java/landl/override.html>
- Oracle. (s. f.). *Documentation comment specification for the standard doclet (JDK 24)*. Recuperado el 14 de marzo de 2026, de <https://docs.oracle.com/en/java/javase/24/docs/specs/javadoc/doc-comment-spec.html>
- Stack Overflow. (2011, febrero 13). *Java - Polymorphism - Overloading/Overriding*. Recuperado el 14 de marzo de 2026, de <https://stackoverflow.com/questions/4986095/polymorphism-overloading-overriding>
- Escuela Colombiana de Ingeniería Julio Garavito. (2026). *POOB-I01-2026-01: Documento del ciclo 1 del proyecto*.
- Escuela Colombiana de Ingeniería Julio Garavito. (2026). *POOB-I02-2026-01: Documento del ciclo 2 del proyecto*.